

SYSTÈME DE GESTION D'UN CABINET MÉDICAL

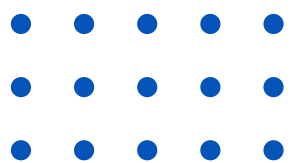
Projet Jakarta EE et UML

Réalisé par :

- **Omaïma Sabri**
- **Assia Houbbadi**
- **Aya El Azzouzi**
- **Khadija El kartouch**

Encadré par :

Mme Karoum Bouchra
Mme Ennaji Fatima Zohra

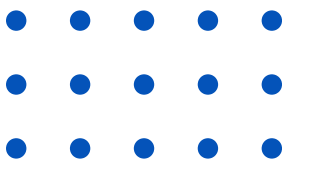


PARTIE 1 : INTRODUCTION & OBJECTIFS

Introduction

Ce projet s'inscrit dans le cadre du module Jakarta EE et UML en spécialité Génie Logiciel. Il vise à appliquer les concepts théoriques à travers le développement d'une application réelle répondant aux enjeux actuels de digitalisation des services de santé. Dans un contexte où les cabinets médicaux nécessitent une optimisation croissante de leurs processus, notre projet permet de passer de la théorie à la pratique en développant une solution complète d'automatisation des tâches administratives et médicales utilisant les technologies modernes étudiées en cours.





Problématique

Les cabinets médicaux rencontrent de nombreuses difficultés dans leur gestion quotidienne lorsqu'ils s'appuient sur des méthodes traditionnelles. La gestion manuelle des dossiers médicaux rend l'accès aux informations lent et expose les données à des risques de perte. La planification des rendez-vous par téléphone génère des erreurs, des doublons et des conflits d'horaires, prolongeant les temps d'attente. La coordination entre le personnel médical et administratif manque de fluidité, entraînant des retards dans la prise en charge des patients. Enfin, la facturation manuelle est chronophage, sujette aux erreurs et complique le suivi financier du cabinet. Face à ces constats, il devient impératif de développer une solution informatique globale garantissant sécurité, efficacité et facilité d'utilisation.

Objectifs du projet

Objectif général

Concevoir et développer un système complet de gestion de cabinet médical permettant d'optimiser l'ensemble des processus administratifs et médicaux.

Objectifs du projet

Objectifs spécifiques

Sur le plan fonctionnel :

- Centraliser la gestion des patients et dossiers médicaux
- Automatiser la planification des rendez-vous
- Digitaliser les consultations et prescriptions
- Simplifier la facturation et le suivi financier
- Créer des tableaux de bord personnalisés par rôle

Sur le plan technique :

- Développer une architecture multi-couches robuste
- Utiliser Java Spring Boot pour le backend
- Créer une interface utilisateur intuitive avec React
- Garantir la sécurité des données médicales (authentification, autorisation)
- Implémenter une base de données PostgreSQL optimisée

Sur le plan méthodologique :

- Appliquer la méthodologie Scrum avec sprints définis
- Produire une modélisation UML complète
- Assurer la qualité par des tests unitaires et d'intégration
- Utiliser Git pour la gestion de version

PARTIE 2 : ANALYSE & CONCEPTION

Analyse des besoins

Acteurs identifiés :

- Administrateur : Gestion technique et validation
- Secrétaire : Accueil et gestion administrative
- Médecin : Suivi médical et prescriptions

Besoins fonctionnels principaux :

- Administrateur : Gestion technique et validation
- Secrétaire : Accueil et gestion administrative
- Médecin : Suivi médical et prescriptions

Contraintes :

- Sécurité des données médicales
- Performance et accessibilité
- Technologies imposées : Spring Boot, React, PostgreSQL

Fonctionnalités principales

Module Administrateur :

- Validation des candidatures de cabinets
- Gestion des utilisateurs et médicaments

Module Secrétaire :

- Enregistrement et gestion des patients
- Planification des rendez-vous
- Facturation

Module Médecin :

- Consultation des dossiers médicaux
- Saisie des diagnostics et observations
- Génération d'ordonnances (médicaments et examens)

Conception du système

Architecture Multi-couches (Backend) :

- Couche Présentation (Controllers REST)
- Couche Métier (Services)
- Couche Accès aux Données (Repositories)
- Modèle (Entities JPA)

Architecture Client-Serveur :

- Frontend React sur port 3000
- Backend Spring Boot sur port 8080
- Base PostgreSQL hébergée sur Supabase

Choix techniques

Pourquoi Spring Boot ?

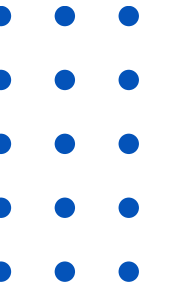
- Framework robuste et mature
- Gestion simplifiée de la sécurité (Spring Security)
- Support natif des API REST

Pourquoi React ?

- Interface dynamique et réactive
- Composants réutilisables
- Écosystème riche

Pourquoi PostgreSQL + Supabase ?

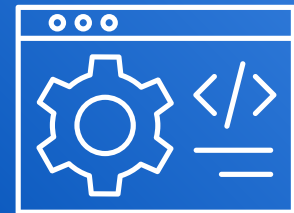
- SGBD relationnel fiable
- Hébergement cloud sécurisé
- Travail synchronisé entre les membres du groupe



PARTIE 3: RÉALISATION & CONCLUSION

Environnement de développement

Pour concrétiser notre conception, nous avons opté pour une stack technologique moderne et éprouvée dans le domaine médical. Notre choix s'est porté sur une architecture client-serveur robuste, combinant un backend Java Spring Boot pour la sécurité et la fiabilité, avec un frontend React pour une expérience utilisateur fluide et accessible.



Technologies Backend

- Java Spring Boot : Framework principal pour l'API REST
- Spring Security + JWT : Authentification sécurisée basée sur les rôles
- Spring Data JPA : Persistance des données
- BCrypt : Hachage sécurisé des mots de passe



Base de données

- PostgreSQL hébergé sur Supabase (cloud)
- Architecture relationnelle avec gestion des transactions atomiques



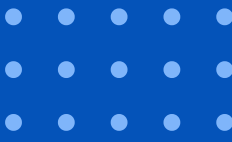
Technologies Frontend

- React.js : Interface utilisateur dynamique et réactive
- Tailwind CSS : Design moderne et responsive
- i18next : Support multilingue (FR, EN, AR avec RTL)



Outils de développement

- Git/GitHub : Versionnement et collaboration
- IntelliJ IDEA : IDE principal
- Postman : Tests des endpoints API
- draw.io , plantuml : Modélisation UML



Réalisation de l'application



Phase 1 : Sécurité & Authentification

- Implémentation du système d'authentification hybride (Email/Login)
- Génération et gestion des tokens JWT
- Processus d'inscription multi-étapes avec validation

Phase 2 : Modules métier

- Secrétaire : Gestion patients, rendez-vous, facturation
- Médecin : Consultations, dossiers médicaux, ordonnances avec autocomplétion
- Admin : Gestion cabinets, utilisateurs, médicaments, candidatures

Phase 3 : Fonctionnalités avancées

- Dark/Light mode avec persistance
- Messagerie interne en temps réel
- Génération PDF d'ordonnances
- Système de notifications



Réalisation de l'application



Organisation du code

Backend : Architecture multicouches

- Controllers (Endpoints REST)
- Services (Logique métier)
- Repositories (Accès données)
- Entities (Modèles JPA)

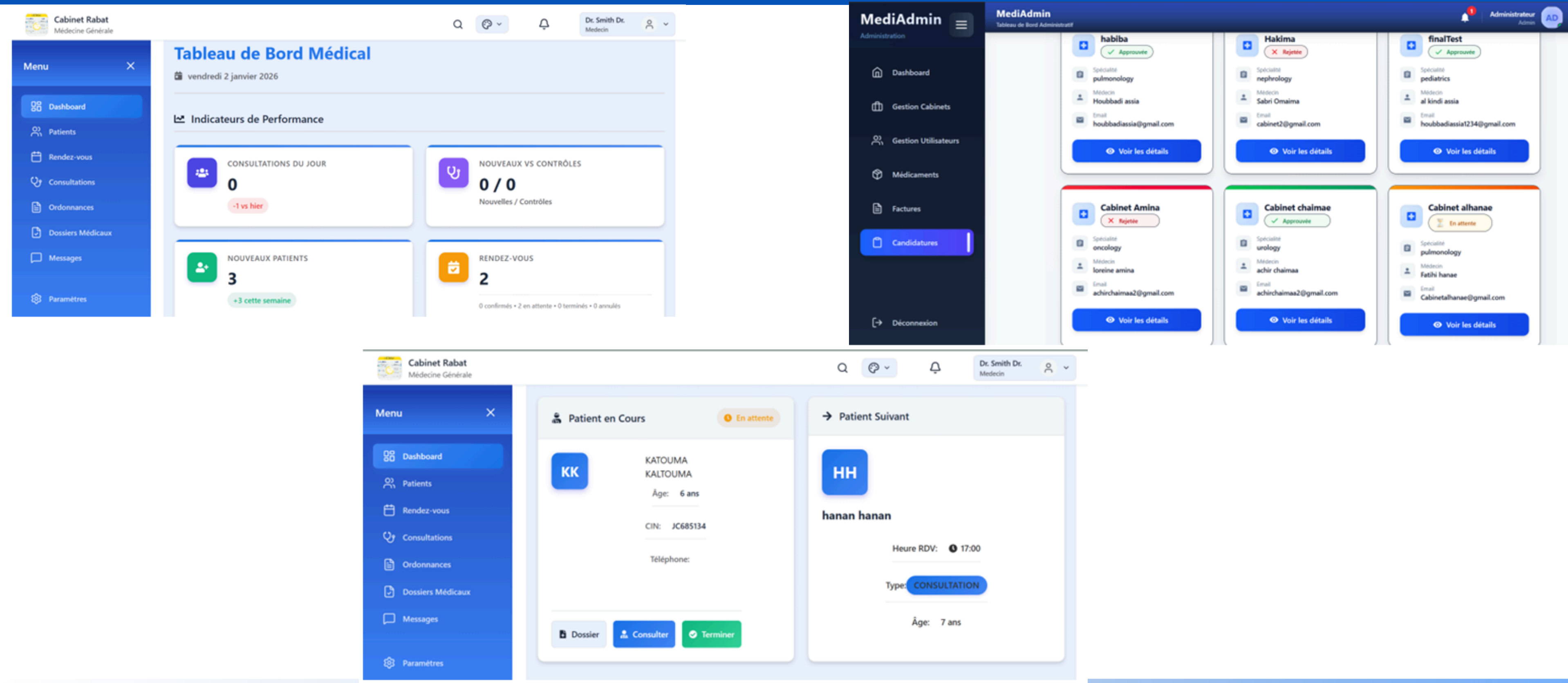
Frontend : Composants React modulaires

- Pages (par rôle)
- Components (réutilisables)
- Services (API calls)

Difficultés rencontrées

- Gestion complexe des états : Cycle de vie des rendez-vous (états + transitions)
- Transactions atomiques : Création simultanée cabinet + utilisateurs + facture
- Sécurité : Validation stricte des accès par rôle et statut cabinet
- RTL pour l'arabe : Adaptation complète de l'interface

Réalisation de l'application



Résultats obtenus

Fonctionnalités réalisées

✓ Processus d'Inscription & Authentification

- Inscription médecin en 4 étapes (Cabinet, Médecin, Secrétaire, Confirmation)
- Validation multi-critères en temps réel
- Authentification hybride (Email/Login) avec sélection de rôle
- Récupération de mot de passe avec token sécurisé
- Gestion des demandes en attente avant validation admin

The screenshot displays a web form titled "Inscription Cabinet Médical" with the subtitle "Gestion Simplifiée et Sécurisée". At the top, a progress bar shows four steps: "Cabinet" (active), "Médecin", "Secrétaire", and "Confirmation". The form fields include:

- Nom du Cabinet ***: A text input field containing "Ex Cabinet Médical Al Chefa".
- Spécialité ***: A dropdown menu with the placeholder text "Sélectionnez une spécialité".
- Téléphone Cabinet ***: A text input field containing "05 XX XX XX XX".
- Email Cabinet ***: A text input field containing "contact@cabinet.com".
- Adresse Cabinet ***: A text input field with the placeholder text "Adresse complète du cabinet".
- Logo du Cabinet**: A file upload area with the text "Choisir un fichier" and a note "(Optionnel)". Below it, a small warning icon states "Format recommandé : PNG ou JPEG (max 2MB)".

At the bottom of the form, there are two navigation buttons: "Précédent" (Previous) on the left and "Suivant" (Next) on the right.

Résultats obtenus

Fonctionnalités réalisées

✓ **Processus d'Inscription & Authentification**

Connexion

Sélectionnez votre rôle:



Administrateur
Accès administrateur



Médecin
Accès cabinet



Secrétaire
Accès secrétariat

Connexion

[← Changer de rôle](#)

Connecté en tant que: **Médecin**

Email ou Identifiant

Mot de Passe

Sélectionnez votre Cabinet

[Mot de passe oublié?](#)

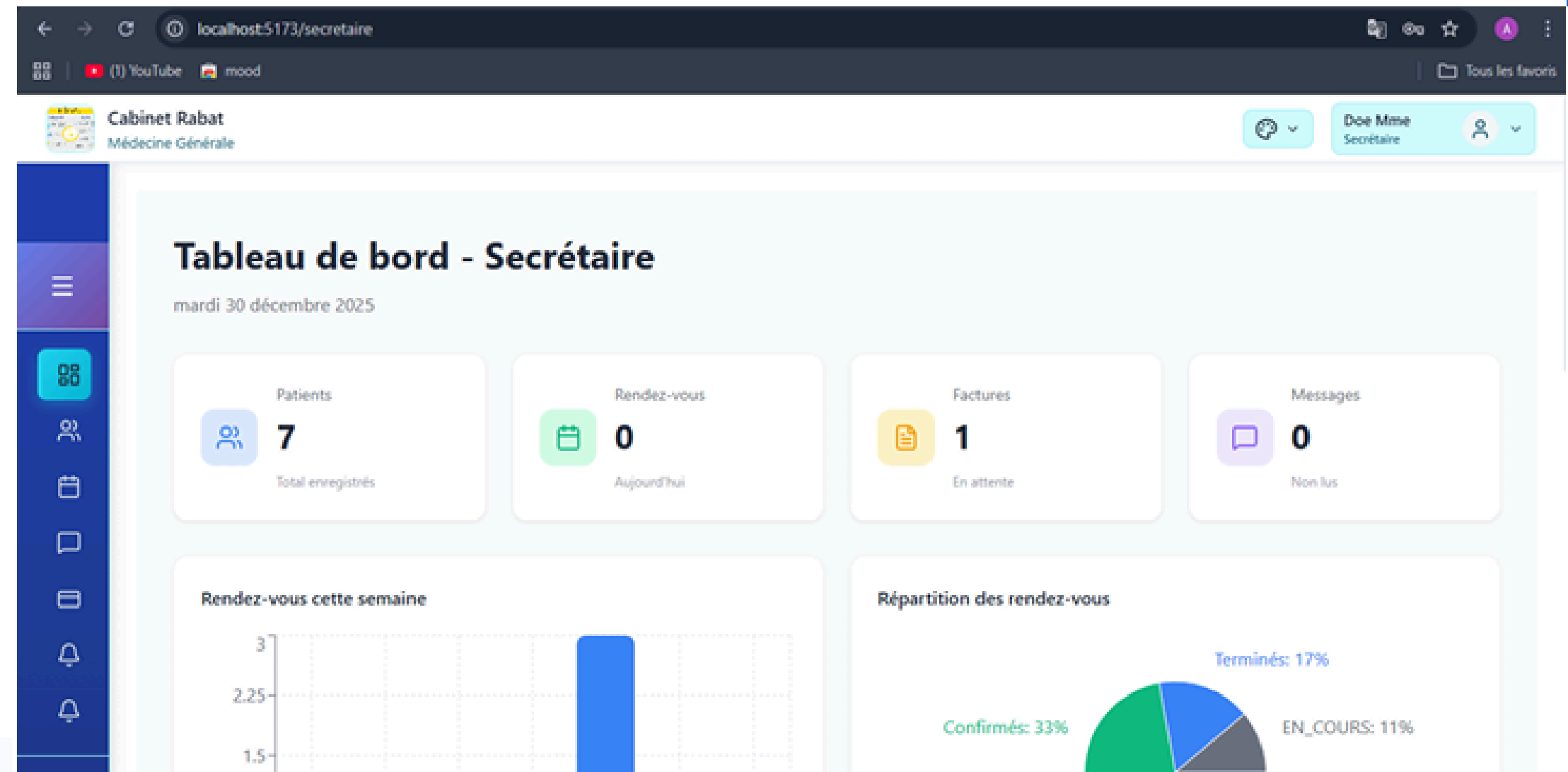
Se Connecter

Résultats obtenus

Fonctionnalités réalisées

✓ Module Secrétaire

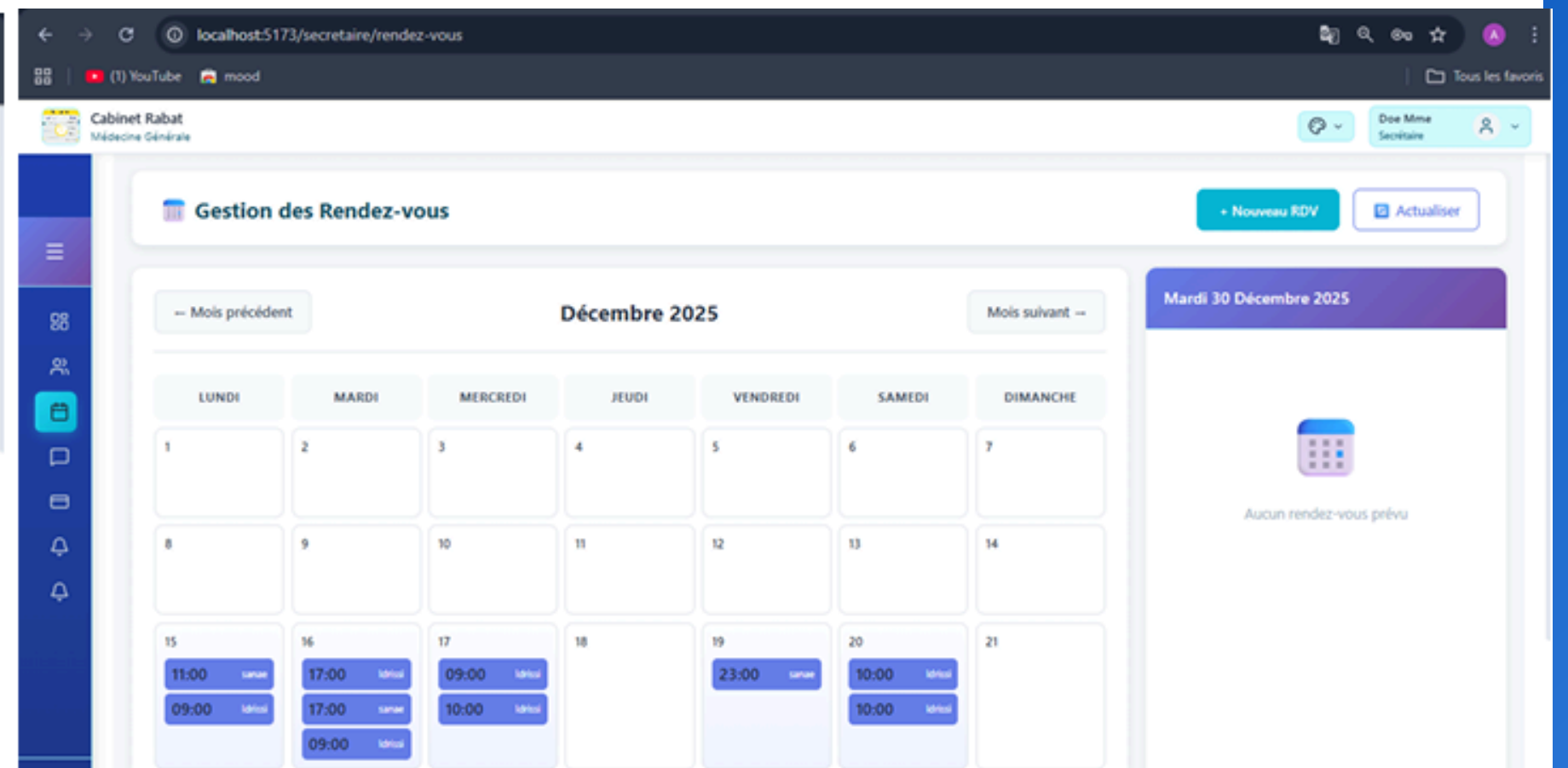
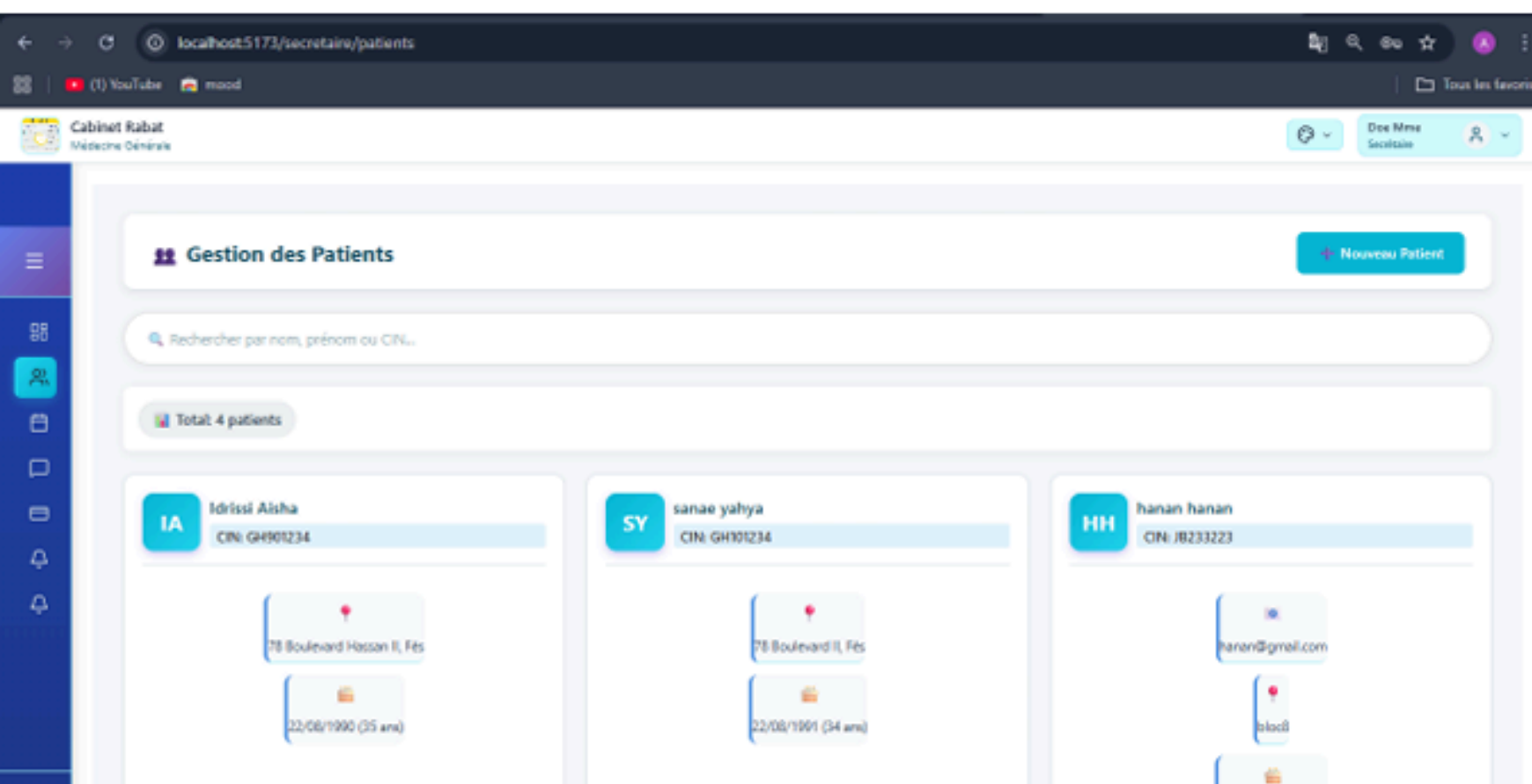
- Gestion complète des patients (CRUD)
- Planification intelligente des rendez-vous avec calendrier
- Facturation automatique avec génération PDF
- Messagerie interne avec support audio



Résultats obtenus

Fonctionnalités réalisées

✓ **Module Secrétaire**



Résultats obtenus

Fonctionnalités réalisées

✓ Module Médecin

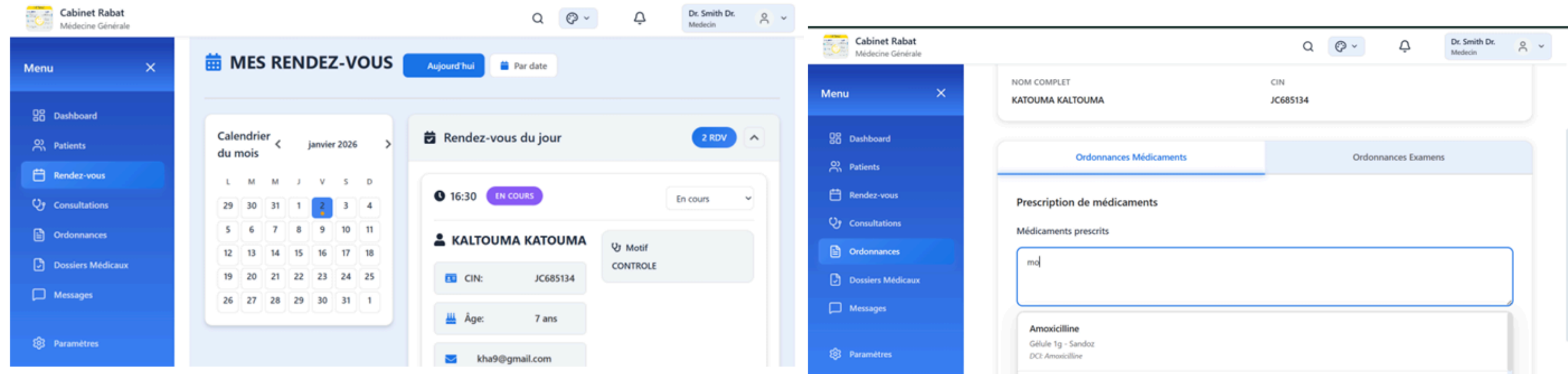
- Dashboard avec statistiques en temps réel
- Gestion consultations avec historique patient
- Prescription d'ordonnances (médicaments + examens)
- Autocomplétion depuis base + médicaments
- Dossier médical électronique complet



Résultats obtenus

Fonctionnalités réalisées

✓ **Module Médecin**

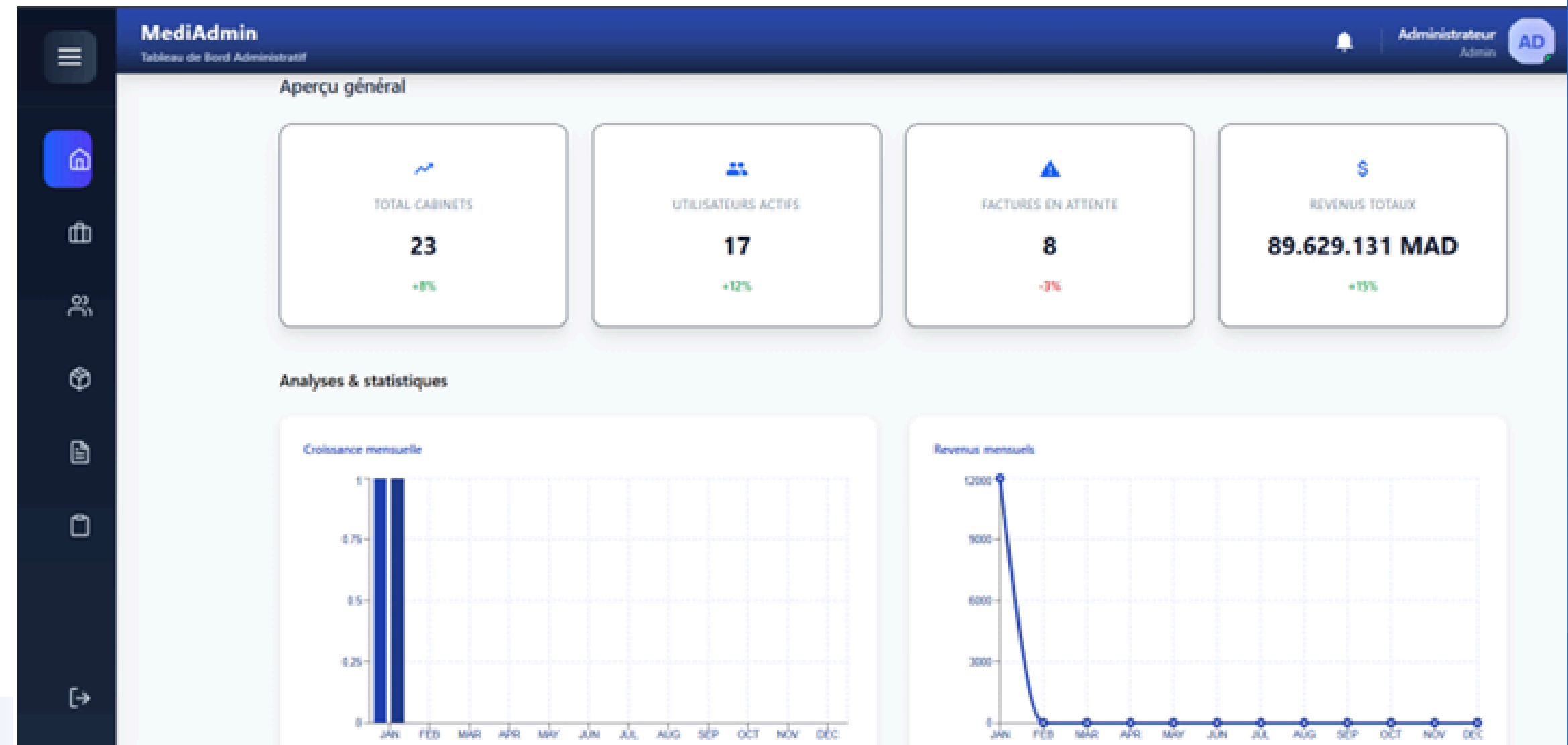


Résultats obtenus

Fonctionnalités réalisées

✓ Module Admin

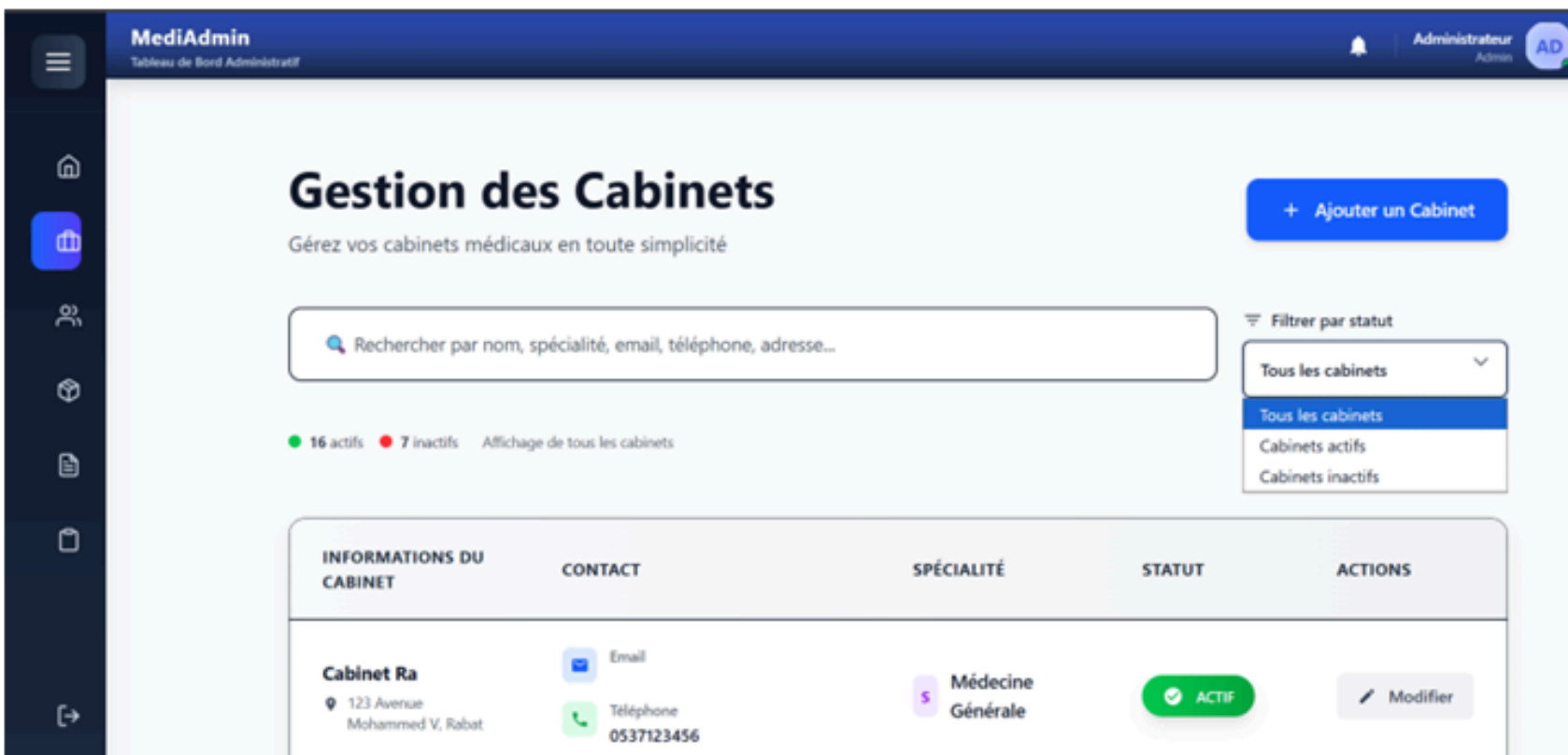
- Validation des candidatures avec documents justificatifs
- Gestion cabinets avec activation/désactivation
- Import CSV massif de médicaments
- Suivi financier



Résultats obtenus

Fonctionnalités réalisées

✓ Module Admin



Résultats obtenus

Objectifs atteints

- ✓ **Sécurité** : Authentification JWT + BCrypt + contrôle d'accès par rôle
- ✓ **Performance** : Réponse API < 200ms, autocomplétion instantanée
- ✓ **Ergonomie** : Interface intuitive, multilingue (3 langues), 2 thèmes
- ✓ **Traçabilité** : Historique complet de toutes les actions

Ce que fait concrètement l'application

1. Un médecin s'inscrit → Admin valide → Cabinet créé automatiquement
2. Secrétaire enregistre un patient → Planifie rendez-vous
3. Médecin consulte → Remplit dossier médical → Prescrit ordonnance
4. Système génère facture → Secrétaire encaisse → Admin suit revenus



Conclusion générale

Ce projet a permis de réaliser une application complète et fonctionnelle de gestion de cabinets médicaux, répondant à un besoin réel de digitalisation. L'ensemble du backlog a été couvert avec succès, avec 15 User Stories implémentées, des interfaces distinctes et opérationnelles, ainsi qu'une architecture robuste basée sur une approche multicouches, une API REST sécurisée et une base de données relationnelle.

La réalisation du projet s'est appuyée sur une méthodologie Agile Scrum, organisée en quatre sprints de deux semaines, favorisant une bonne planification, une collaboration efficace et une traçabilité claire du travail grâce à des outils comme GitHub et Notion.

Sur le plan pédagogique, ce projet a renforcé notre maîtrise de Spring Boot, Spring Security, JPA, des API REST, ainsi que le développement d'un frontend moderne avec React. Il nous a également permis de consolider nos compétences en modélisation UML, en gestion de bases de données et en sécurisation applicative.

Au-delà des aspects techniques, cette expérience a développé des compétences essentielles telles que le travail en équipe, la gestion du temps, l'adaptabilité et la résolution de problèmes complexes. Ce projet constitue ainsi une expérience complète d'ingénierie logicielle et une base solide pour des projets professionnels futurs.

Merci pour votre attention. Des questions ?



Merci

Pour votre attention